

Amortized PEFT-Conditioned Data Selection for Cross-PEFT Fine-Tuning

Anonymous submission

Abstract

Parameter-efficient fine-tuning (PEFT) is rarely deployed as a single configuration: a lab or platform maintains a registry of LoRA ranks, module placements, adapters, (IA)³, and other recipes chosen for different tasks and deployment budgets. Gradient- and influence-based data selectors produce strong subsets but bind to the target PEFT—each new configuration rebuilds a target-specific gradient datastore—so selection cost grows linearly with the registry. Reusing one configuration’s selected subset removes that cost but sacrifices quality, because a subset optimized for one trainable subspace is mismatched to the others.

We present PCU-Select, which recasts this per-target recomputation as reusable supervision. It models example utility as a quantity conditioned jointly on the task and a structured PEFT code, builds a shared representation of samples, tasks, and PEFT configurations once offline, and trains a conditional scorer with multi-fidelity utility labels. Transferring to a new target PEFT requires only encoding its PEFT code and task sketch and one forward pass over cached sample features—no per-target gradient datastore.

Across Llama-2-7B, four tasks, and five seen PEFT configurations, PCU-Select improves consistently over PEFT-agnostic selectors and near-ties a per-PEFT LESS reimplementation on aggregate downstream quality while still producing distinct per-target subsets. Its contribution is amortization, not a single-configuration win: after 72.0 GPU-hours of offline supervision, each PEFT–task application costs 0.18 GPU-hours. Serving five PEFTs on four tasks therefore costs 75.6 GPU-hours, versus 158.0 for per-PEFT LESS, and breaks even after 2.33 configurations. A single reused LESS subset falls below the PEFT-agnostic frontier. Near-support configurations transfer zero-shot, while far same-family targets and BitFit benefit from calibration; Prefix/P-Tuning remain a zero-shot failure boundary.

1 Introduction

Adapting large language models under limited compute requires efficiency on both the model side and the data side. A lab or platform rarely maintains a single adaptation setting; it keeps a *registry* of parameter-efficient fine-tuning

(PEFT) configurations—LoRA at several ranks and placements, adapters, (IA)³, prefix tuning, or BitFit—chosen for different tasks, latency targets, memory limits, and deployment constraints. PEFT freezes the backbone and trains a small set of task-specific parameters, reducing optimizer memory and deployment storage. Data-efficient fine-tuning is the complementary tool: it selects a compact subset from a larger instruction pool, reducing the number of examples and training steps. These two tools are usually applied independently: PEFT controls which parameters move, while data selection controls which examples drive the update.

Current data selectors rarely model this interaction. Quality filters, diversity heuristics, proxy-model scores, and gradient-similarity methods often assign each example a single value for a target task and then reuse the selected subset across adaptation methods. That assumption is convenient, but PEFT methods change the trainable subspace itself. LoRA injects low-rank additive updates into selected projections (Hu et al. 2022); adapters insert residual bottlenecks (Houlsby et al. 2019); and (IA)³ learns multiplicative activation scalings (Liu et al. 2022). An example whose gradient is aligned with a task inside one subspace need not be aligned inside another.

When the same pool must serve a whole registry, this omission becomes a cost problem. The strongest selectors are gradient- and influence-based: LESS (Xia et al. 2024) builds a target-specific gradient datastore and ranks candidates by their influence on a validation sketch. Because that datastore is tied to the target adapter’s own trainable parameters, every new configuration pays to rebuild it, and selection cost grows linearly with the number of PEFTs served. The obvious shortcut—run the selector once and reuse its subset for every configuration—removes the cost but forfeits the quality that motivated a gradient selector: a subset chosen for one trainable subspace is mismatched to the others, and we measure that reusing a single LESS subset falls below even a PEFT-agnostic representation baseline (Table 2). This is the reuse-versus-recompute dilemma at the center of the paper: existing methods are either expensive per target or cheap but PEFT-agnostic in effect. The expensive supervision should instead be computed *once* and reused across configurations.

Our motivation study shows that PEFT-specific utility changes with the trainable subspace. We estimate short-horizon utility for the same candidate pool under LoRA,

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

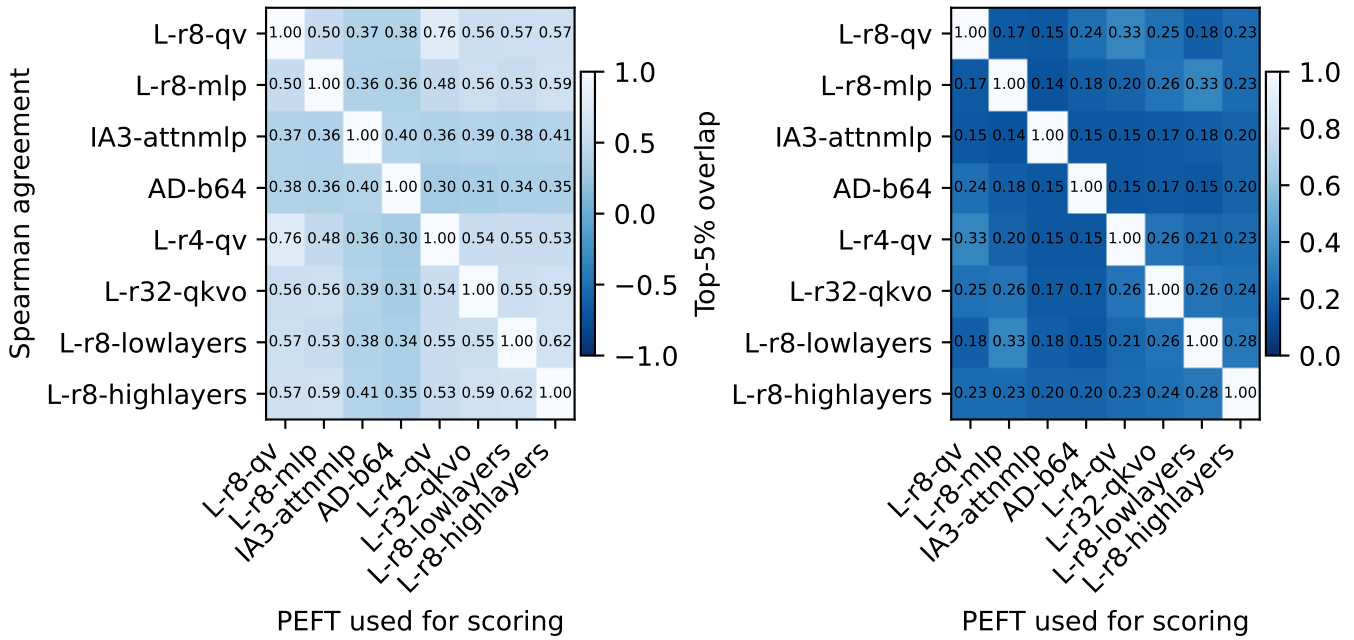


Figure 1: Short-horizon data-value estimates disagree across PEFT configurations. A common pool of 20K candidate examples is scored under each target PEFT with two task sketches and independent anchor/seed replicates; each cell is the Spearman rank correlation between the row and column scoring runs. The heatmap diagonal is self-correlation (1.00 by construction); *independent* same-PEFT replicates agree only moderately ($\rho = 0.79$, Top-5% overlap = 0.32), reflecting the short-horizon label-noise floor against which cross-PEFT agreement should be read. Cross-PEFT agreement falls further with structural distance: cross-family pairs average $\rho = 0.36$ and Top-5% overlap = 0.17.

adapter, and (IA)³ variants, using two task sketches and independent anchor/seed replicates. Figure 1 shows that within-PEFT agreement exceeds cross-PEFT agreement. Same-family capacity changes retain moderate structure, but placement changes and cross-family changes sharply reduce rank agreement and Top-5% overlap. A transfer matrix gives the downstream consequence: selecting data for the same PEFT used in training beats mismatched *cross-family* selection by 2.0 points on average across target PEFTs (Appendix Table 15); the Analysis section reports the narrower 1.45-point within-LoRA configuration-scan gap.

This paper treats data value as a PEFT-conditioned quantity. We ask: given a candidate example x , a target task t , and a target PEFT configuration p , can we predict whether x will help when the trainable parameters are exactly those exposed by p ? This framing differs from PEFT-agnostic selection, which asks whether x is generally good for the task. It also differs from recomputing an influence selector for every PEFT, because the expensive supervision should be reused across many target configurations.

PCU-Select (PEFT-Conditional Utility) addresses this question with an intervention-site utility layer. It maps samples, task sketches, and PEFT configurations into a shared set of 24 module-output sites in the backbone. Sample and task signatures describe gradient alignment at each site. The PEFT code describes which sites a method touches, how it acts there, and how much trainable capacity it has. PCU-Select uses a low-fidelity site-weighted gradient proxy for

broad coverage, corrects it with short-horizon high-fidelity labels, and trains a conditional scorer that predicts both utility and uncertainty. At deployment, the scorer ranks the pool for a target (p, t) and a quota allocator balances utility with semantic coverage.

We make three contributions.

- **Problem.** We formulate cross-PEFT data-efficient fine-tuning as quality-preserving amortization: reusable supervision should match per-target influence selection while avoiding repeated datastore construction. Data-value rankings differ across configurations (cross-family Top-5% overlap 0.17; mismatch penalty 2.0 points), and reusing one influence-selected subset drops below a PEFT-agnostic baseline.
- **Method.** PCU-Select turns intervention-site signatures, structured PEFT codes, task sketches, and multi-fidelity labels into reusable target-specific rankings. Removing the PEFT code loses 1.37 points, while removing task relativity loses 0.82, validating the two conditions.
- **Evaluation.** Across GSM8K, HumanEval, MMLU, and TyDiQA on a fixed Llama-2-7B backbone (Touvron et al. 2023), PCU-Select improves 10%-budget downstream quality over PEFT-agnostic selectors (over random by 2.89 points and RDS+ by 0.74 points) and near-ties a per-PEFT LESS reimplementation on the seen registry while still generating distinct per-target subsets. Its offline supervision amortizes after 2.33 target configurations; serving

five PEFTs on all four tasks costs 75.6 GPU-hours, versus 158.0 for LESS (2.09 $\times$).

2 Related Work

Instruction-data selection. Instruction tuning benefits from small but carefully selected data. LIMA shows that 1,000 curated examples can produce strong instruction-following behavior in a large model (Zhou et al. 2023). AlpGasus filters low-quality Alpaca examples with LLM judgments (Chen et al. 2024); InsTag measures semantic diversity and complexity through instruction tags (Lu et al. 2023); and DEITA combines quality, complexity, and diversity criteria for automatic data selection (Liu et al. 2024). Other selectors emphasize explicit diversity through quality-diversity trade-offs, log-determinant objectives, or partition-aware sampling (Bukharin et al. 2023; Wang et al. 2024; Rao et al. 2024). These methods improve data efficiency, but they mostly treat value as a property of the example and target task rather than of the PEFT subspace used for adaptation.

Proxy-model and influence selection. Model-assisted selectors score examples with external judges, weak models, or training signals. Cherry LLM uses instruction-following difficulty (Li et al. 2023); Superfiltering studies when weak models can rank data for stronger models (Li et al. 2024); and SmallToLarge transfers trajectory information from small models to larger targets (Yang et al. 2024). Influence-style methods estimate how training examples affect validation behavior. Classical influence functions use second-order approximations (Koh and Liang 2017), TracIn accumulates gradient similarities along checkpoints (Pruthi et al. 2020), and LESS builds a low-dimensional gradient datastore for instruction selection (Xia et al. 2024). DataInf specializes influence approximation for LoRA-tuned generative models (Kwon et al. 2024). These methods recompute target-specific gradients or influence for every new configuration. PCU-Select instead conditions a learned scorer on a structured PEFT code, so one offline supervision serves many targets and the expensive per-target recomputation is amortized rather than repeated; we compare against both a per-PEFT LESS reimplement and a cheap shared-datastore Influence baseline.

Parameter-efficient fine-tuning. PEFT methods reduce adaptation cost by updating a small module or parameter subset. Adapters insert bottleneck modules into Transformer layers (Houlsby et al. 2019; Pfeiffer et al. 2021). Prefix and prompt tuning optimize continuous prompts or prefix states (Li and Liang 2021; Lester, Al-Rfou, and Constant 2021). LoRA learns low-rank additive updates (Hu et al. 2022), QLoRA combines quantization with LoRA for memory efficiency (Dettmers et al. 2023), BitFit tunes only bias terms (Ben Zaken, Goldberg, and Ravfogel 2022), and (IA)³ rescales attention and feed-forward activations (Liu et al. 2022). PEFT work usually compares accuracy, parameter count, memory, or inference overhead under fixed data. This paper asks a complementary question: when the candidate pool is fixed, do different PEFT methods assign high utility to the same examples?

Joint data- and parameter-efficient adaptation. Data selection and PEFT address two sides of efficient adaptation, but most work optimizes one side while holding the other fixed. DataInf and LESS move toward the intersection by using gradients from LoRA-style training (Kwon et al. 2024; Xia et al. 2024), and concurrent geometry-aware targeted selection suggests that PEFT optimization has nontrivial subspace structure (Min et al. 2026). These still tie the selector to one optimization geometry. We use LESS as the matched baseline because its target-specific datastore exposes the re-computation cost central to our claim; DataInf and concurrent GIST study different estimators. PCU-Select instead treats the *structured PEFT configuration itself*—its sites, operator, capacity, and recipe—as an explicit condition and amortizes supervision across configurations.

3 Problem Formulation

We formalize *cross-PEFT data-efficient fine-tuning*: selecting a compact training subset whose value depends on the target PEFT configuration and target task. Fix a backbone family and frozen base model M_0 . A candidate pool $\mathcal{D} = \{x_i\}_{i=1}^N$ contains instruction-response examples. A task t provides a small validation sketch V_t drawn from train or development data, never from the test set. A PEFT configuration $p \in \mathcal{P}$ specifies the family, trainable locations, capacity, and training recipe. Given budget B , the ideal subset solves

$$S^* = \arg \max_{S \subseteq \mathcal{D}, |S| \leq B} \text{Perf}(\text{Tune}(M_0, p, S), V_t^{\text{test}}). \quad (1)$$

The held-out set V_t^{test} appears only in this definition of the oracle; it is never used for training or for selection, which see only the sketch V_t . Directly optimizing Eq. (1) is infeasible because it requires training and evaluating many subsets for every target (p, t) pair.

PCU-Select replaces the subset oracle with a conditional per-example utility,

$$s_\phi(x, p, t) \approx u(x, p, t), \quad (2)$$

where u measures how much x helps task t when the trainable parameters are those of p . Concretely, $u(x, p, t) \in [0, 1]$ summarizes sketch-loss reductions into a rank-normalized short-horizon utility score, as defined in the Method section. Conditioning on p is the core departure from PEFT-agnostic selection. A sample’s gradient can be useful in the attention projections trained by LoRA, weak in the residual bottlenecks trained by adapters, and different again under multiplicative (IA)³ gates. The selected subset should therefore change when the trainable subspace changes.

Amortization objective. A registry makes cost, not single-target accuracy, the operative criterion. Let P be the number of target PEFT configurations and Q the number of task sketches served from a fixed pool. LESS builds one datastore per PEFT and ranks it against each task, whereas PCU-Select pays one offline cost and one scorer application per PEFT-task pair:

$$\begin{aligned} C_{\text{LESS}}(P, Q) &= P(C_{\text{LESS}}^{\text{data}} + QC_{\text{LESS}}^{\text{rank}}), \\ C_{\text{PCU}}(P, Q) &= C_{\text{offline}} + PQ C_{\text{PCU}}^{\text{pair}}, \end{aligned} \quad (3)$$

with $C_{\text{PCU}}^{\text{pair}} \ll C_{\text{LESS}}^{\text{data}}$. The goal is *quality-preserving amortization*: match the per-PEFT selector’s downstream quality,

$$\Delta_{\text{quality}} = \text{Perf}_{\text{PCU}} - \text{Perf}_{\text{LESS}} \approx 0, \quad (4)$$

while spending less once P exceeds $P^* = C_{\text{offline}} / [C_{\text{LESS}}^{\text{data}} + Q(C_{\text{LESS}}^{\text{rank}} - C_{\text{PCU}}^{\text{pair}})]$. Under this criterion a near-tie with LESS is the design target: LESS is the quality reference and PCU-Select is its amortized alternative. Reusing one LESS-selected task subset per task pays one datastore plus Q rankings but removes target-PEFT conditioning, which lowers quality.

Task relativity is concrete: a worked-solution arithmetic example receives high utility under the GSM8K sketch but low utility under the TyDiQA sketch, because its gradient aligns with the reasoning task but not with multilingual span extraction; the same example’s utility also shifts across PEFTs when the aligned sites are not the ones a given method trains.

The formulation rests on four requirements. **PEFT dependence**: the scorer must take p as an explicit input. **Task relativity**: utility must be defined relative to V_t so that the model does not learn a generic quality score. **Structural encodability**: PEFT methods must be represented by their sites, operator type, capacity, and recipe, not just by a family name. **Amortizable cost**: expensive labels and scorer training are paid offline, while applying to a new (p, t) pair uses cached features and forward scoring.

4 Method: PCU-Select

PCU-Select learns the surrogate in Eq. (2) and uses it to select a budgeted subset for a target PEFT and task. Figure 2 summarizes the pipeline. The offline stage extracts reusable sample and task signatures, builds low- and high-fidelity utility labels, and trains the scorer. The online stage encodes a target (p^*, t^*) , assigns its preregistered structural support tier, scores the pool, and allocates the budget across semantic clusters. Algorithms 1 and 2 in the appendix state both stages.

Intervention Sites

PCU-Select compares PEFT methods through module-output sites rather than parameter tensors. It hooks attention, MLP, and post-residual outputs at eight evenly spaced Transformer layers, giving $|\Omega| = 24$. By the chain rule, a PEFT parameter gradient factors through the gradient at the corresponding module output; these shared coordinates also avoid fused-versus-split projection shapes. The post-residual coordinate represents adapters alongside LoRA and (IA)³. Appendix Table 12 varies layer coverage, site type, and projection dimension: the default is within 0.08 points of larger site spaces while costing up to $4\times$ less.

Each PEFT configuration maps to a site mask and an operator type. Attention LoRA and (IA)³-attention touch attention-output sites; MLP LoRA and (IA)³-FFN touch MLP-output sites; adapters touch block-residual sites; prefix tuning touches attention-output sites; and BitFit is represented as a bias shift. PCU-Select assigns each site a normal-

ized PEFT weight

$$\begin{aligned} w_p^\omega &= \text{mask}(p, \omega) c(p, \omega), \\ c(p, \omega) &= \tanh(\eta \log(1 + \text{cap}(p, \omega)/d_{\text{model}})), \\ \tilde{w}_p^\omega &= w_p^\omega / \max(\epsilon, \sum_{\omega'} w_p^{\omega'}). \end{aligned} \quad (5)$$

This weight is zero for untouched sites and increases with capacity at touched sites. The mapping in Appendix Table 10 places LoRA rank, adapter width, (IA)³ vector count, prefix length, and BitFit bias count on a trainable-parameter scale. The tanh prevents high-capacity configurations from dominating. Operator type is encoded separately in z_p ; a family-wide scalar would cancel under the normalization in Eq. (5).

Representations and Utility Labels

A sample representation $z_x = [e_x; d_x; a_x] \in \mathbb{R}^{848}$ concatenates a 768-dimensional semantic embedding, 16 difficulty and format statistics, and a 64-dimensional activation signature. The semantic vector is the frozen `sentence-transformers/all-mpnet-base-v2` joint instruction-response embedding (Song et al. 2020; Reimers and Gurevych 2019); we do not train this encoder on the candidate pool or target benchmarks. The difficulty vector contains instruction length, response length, total length, response/instruction ratio, seven response-LM statistics (mean/std/max loss, perplexity, average log-probability, mean/max entropy), and five format flags (reasoning marker, code marker, question marker, English, Chinese). The activation signature mean-pools response-token hidden states at the eight sampled layers and projects them to 64 dimensions.

A task sketch V_t is encoded to $z_t \in \mathbb{R}^{848}$ by pooling the same sample representation over 32 sketch examples and averaging their site gradient signatures. The reported PEFT code $z_p = [m_p; c_p; r_p] \in \mathbb{R}^{128}$ concatenates a 24×4 site/operator mask (m_p , 96 dims), a 16-dimensional capacity vector (c_p), and a 16-dimensional training-recipe vector (r_p). The mask channels encode active sites, additive updates, multiplicative scaling, and prefix-style control. Capacity slots distinguish LoRA, adapters, (IA)³, prefix-style methods, and BitFit; recipe features encode module count, placement, and optimizer settings. At the 24-site granularity, q/v and q/k/v/o LoRA both touch attention-output sites, so capacity and recipe features retain their projection-level distinction.

Prefix, P-Tuning, and BitFit use the same deterministic site, operator, capacity, and recipe rules as the seen families. The reported model relies exclusively on these structural features, without a learned family lookup or operator centroid. Zero-shot behavior therefore measures structural transfer; where native short-horizon labels exist, calibration fits the residual head described below.

Low-fidelity utility aligns each sample with the target sketch on the sites trained by the target PEFT:

$$u^{\text{lo}}(x, p, t) = \sum_{\omega \in \Omega} \tilde{w}_p^\omega \cos(g_x^\omega, g_t^\omega). \quad (6)$$

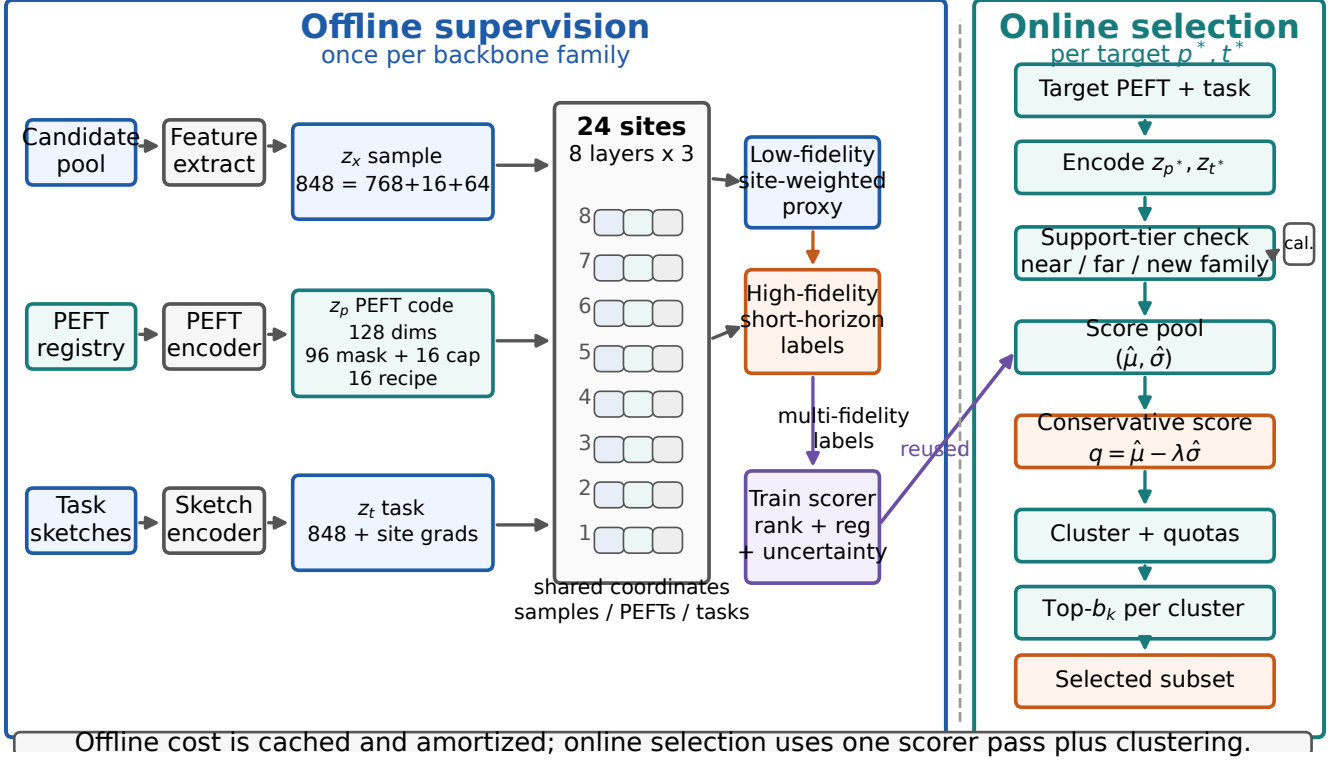


Figure 2: PCU-Select separates reusable offline supervision from online PEFT–task scoring. Intervention sites Ω link sample gradients, task sketches, and PEFT configurations. The reported PEFT code is deterministic and 128-dimensional: a 96-dimensional site/operator mask, 16 capacity dimensions, and 16 recipe dimensions.

Here g_x^ω is a unit-normalized Johnson–Lindenstrauss random projection (Johnson and Lindenstrauss 1984) of the response-token loss gradient at site ω , and g_t^ω is the sketch mean. All task sketches use the same response-token cross-entropy for signatures; task-native metrics such as option likelihood, Pass@1, and F1 are used only for downstream evaluation. The cosine proxy can be negative, so PCU-Select rank-normalizes u^{lo} within each (p, t) bucket before proxy distillation and ranking losses. The proxy is cheap after sample signatures are cached, so it labels many (x, p, t) triples.

High-fidelity utility corrects the proxy near the selection boundary. For a sampled triple, PCU-Select performs short PEFT updates from a base anchor and a warm anchor, then measures sketch-loss reduction:

$$\Delta(x, p, t, a, h) = L_{V_t}(\theta_a) - L_{V_t}(\text{Adapt}_p^h(x; \theta_a)). \quad (7)$$

Rank-normalized reductions from horizons $h \in \{1, 4\}$ and both anchors define u^{hi} . The high-fidelity budget is 10K triples, split across coverage sampling, uncertainty sampling, and boundary sampling. Each high-fidelity update uses the sampled example plus seven in-bucket fillers; labels therefore estimate a stable mini-batch marginal, not an isolated single-example gradient. Coverage and uncertainty labels are generated first, a provisional scorer is trained, and boundary sampling then targets scorer-vs-label rank disagreements from that provisional pass. Rank normalization is always within

the same PEFT, task, anchor, and horizon bucket.

Scorer and Selection

The scorer predicts $(\hat{\mu}, \hat{\sigma}) = s_\phi(z_x, z_p, z_t)$. Three encoders map z_x , z_p , and z_t into hidden vectors. The PEFT and task codes modulate the sample code through FiLM (Perez et al. 2018), while a bilinear term models sample-PEFT interaction. Training combines high-fidelity ranking and regression losses, low-fidelity proxy distillation, and a heteroscedastic uncertainty loss. Stage A pretrains on the proxy for broad conditional structure; stage B enables all losses and selects checkpoints on held-out PEFT/task pairs. We fix the sketch size, label allocation, anchors, horizons, and scalar selection hyperparameters on these held-out pairs before downstream evaluation.

At application time, PCU-Select ranks examples by a conservative score

$$q(x) = \hat{\mu}(x, p^*, t^*) - \lambda \hat{\sigma}(x, p^*, t^*), \quad \lambda = 0.2. \quad (8)$$

Pure top- k ranking can over-select near-duplicates, so PCU-Select clusters examples in semantic space and allocates budget across clusters:

$$b_k = \text{round} \left(B \frac{(v_k^+)^{\gamma} |C_k|^{1-\gamma}}{\sum_j (v_j^+)^{\gamma} |C_j|^{1-\gamma}} \right), \quad \gamma = 0.6. \quad (9)$$

Here v_k is the mean conservative score of the top 10% examples in cluster C_k , and $v_k^+ = \max(v_k, 0)$. For the 300K pool, $k = \max(50, \lfloor \sqrt{N} \rfloor)$ yields 547 clusters with about 548 examples per cluster on average; clustering is included in the 0.18 GPU-hours per PEFT-task application reported below. The final subset takes the top- b_k examples by q inside each cluster. Clusters with non-positive value receive no budget; deterministic remainder handling makes the quotas sum to B . For unseen configurations, a registry-defined structural policy distinguishes near-support, far-support, and unseen-family targets. Far targets use an optional residual calibration head when the native high-fidelity labeler supports their PEFT family.

5 Experiments

The evaluation asks four questions: does PCU-Select improve selected-data performance, does it amortize cost across PEFTs, which components matter, and how does it behave on unseen configurations?

Setup

We evaluate on GSM8K, HumanEval, MMLU, and TyDiQA (Cobbe et al. 2021; Chen et al. 2021; Hendrycks et al. 2021; Clark et al. 2020). Metrics use each task’s native scale: exact match, Pass@1, accuracy, or F1. The candidate pool contains 300K mixed instruction examples, and the main selection budget is 10%; budget sensitivity also uses 5% and 30%. Each method trains the same target backbone, PEFT recipe, number of steps, and evaluation protocol; only the selected subset changes. Main-table dispersion varies target-training seeds 0, 1, and 2 while fixing selector, scorer, sketch, and pool state at seed 0. Appendix Table 8 lists the full 17-configuration registry.

The five seen configurations cover attention and MLP LoRA, (IA)³, and adapters; held-out tests vary rank, placement, learning rate, and family. We compare Random, heuristic and representation selectors, RDS+, and two gradient baselines; Appendix A gives full specifications and omitted aggregate rows. **RDS+** ranks frozen external semantic embeddings against the task-sketch mean and is PEFT-agnostic. **Influence** uses projected base-model gradients in one shared datastore. **LESS** is our per-PEFT implementation of gradient-influence selection (Xia et al. 2024): it warms up each target adapter, builds a target-specific Adam-preconditioned gradient datastore, and ranks by sketch influence. PCU-Select uses its default conditional scorer, uncertainty penalty, and adaptive quotas.

Main Results

PCU-Select averages 35.18, versus 32.29 for Random, 34.44 for RDS+, 34.68 for Influence, and 35.14 for LESS. Across 20 PEFT×task cells, its mean gains are 0.74 over RDS+ (descriptive fixed-state interval $[+0.38, +1.06]$; 17/20 wins) and 0.50 over Influence ($[+0.21, +0.75]$; 18/20 wins). Its 0.04-point gap to LESS has interval $[-0.26, +0.33]$ and 10/20 wins; task-stratified resampling gives $[-0.15, +0.22]$. These summaries describe shared-state cell variation, not independent-sample uncertainty.

The LESS comparison is task-dependent: PCU-Select gains on MMLU, is within seed dispersion on GSM8K and HumanEval, and trails on TyDiQA and MLP-only LoRA. Appendix Tables 3–5 report the unaveraged and normalized views. **Takeaway.** PCU-Select improves over PEFT-agnostic selection and near-ties per-PEFT LESS through reusable supervision.

Held-out utility rankings support the mechanism: PCU-Select reaches Spearman 0.625, NDCG@K 0.703, and pairwise accuracy 0.793, modestly ahead of LESS (Appendix Table 14). Its advantage is largest at a 5% budget and narrows by 30% (Table 13).

Reuse vs. Recompute

Table 2 tests the cheap alternative to recomputation: run LESS once on L-r8-qv and reuse that subset across all five targets. This **Reuse-one-LESS** condition costs 31.6 GPU-hours.

Reuse-one-LESS averages 34.24, 0.90 points below per-PEFT LESS and below RDS+ (34.44). It matches LESS on its source but loses 0.52–1.47 points elsewhere, with larger losses when trainable sites or family change (Appendix Table 17). Reusing a gradient-selected subset therefore discards the condition that made it valuable.

PCU-Select instead produces a distinct subset for each target and preserves LESS-level quality (35.18 vs. 35.14). Five PEFTs over four tasks cost 75.6 GPU-hours, versus 158.0 for LESS. **Takeaway.** PEFT-agnostic methods are cheaper, while PCU-Select occupies the quality-preserving amortized point.

Amortized Cost

On $8 \times A100$ -80GB, PCU-Select pays 72.0 GPU-hours offline and 0.18 per (p, t) application. LESS pays 31.0 per PEFT datastore plus 0.15 per task, or 31.6 per four-task configuration. Equation (3) gives $P^* = 72.0 / [31.0 + 4(0.15 - 0.18)] = 2.33$ configurations; Appendix Figure 4 gives the full curve. The shared 6.4 GPU-hour downstream run is excluded for every selector.

Five hundred calibration labels add 0.525 GPU-hours per (p, t) , or 2.1 per four-task PEFT; worst-case calibration shifts break-even to 2.50 configurations. Prefix/P-Tuning lack native calibration labels. **Takeaway.** Random and RDS+ remain preferable when raw selection cost outweighs the quality gain; PCU-Select amortizes after 2.33 configurations when quality preservation matters.

6 Analysis

PEFT Conditioning Produces Target-Specific Subsets

Across the LoRA scan, matched conditioning beats reuse from another configuration by 1.45 points. PCU-Select averages 21.25, versus 20.70 for LESS and 20.27 for RDS+. Its 21 unordered configuration pairs have mean Jaccard 0.427; PEFT-agnostic RDS+ is exactly 1.000. Configuration distance correlates with selection difference at descriptive Spearman $\rho = 0.969$ (leave-one-configuration-out range

Table 1: Downstream performance at a 10% selection budget (mean $\pm$ SD over three matched target-training seeds; four task-native scores per PEFT). Selector, sketch, and scorer state are fixed. PCU-Select near-ties per-PEFT LESS: the paired difference over the 20 PEFT $\times$ task cells is 0.04 points with a descriptive cell-resampling interval $[-0.26, +0.33]$. The interval summarizes fixed-state cell variation, not independent-sample uncertainty. Boldface marks the best mean.

Method	AD-b64	IA3-attnmlp	L-r16-qkvo	L-r8-mlp	L-r8-qv	Avg.
Random	32.79 $\pm$ 0.15	31.74 $\pm$ 0.08	33.11 $\pm$ 0.23	31.97 $\pm$ 0.23	31.82 $\pm$ 0.11	32.29 $\pm$ 0.11
RDS+	35.27 $\pm$ 0.18	33.54 $\pm$ 0.31	35.00 $\pm$ 0.31	34.46 $\pm$ 0.35	33.91 $\pm$ 0.32	34.44 $\pm$ 0.11
Influence	35.08 $\pm$ 0.22	33.84 $\pm$ 0.15	35.24 $\pm$ 0.25	34.69 $\pm$ 0.25	34.53 $\pm$ 0.32	34.68 $\pm$ 0.16
LESS	35.52 $\pm$ 0.09	34.44 $\pm$ 0.12	35.80 $\pm$ 0.25	35.13$\pm$0.11	34.80 $\pm$ 0.40	35.14 $\pm$ 0.16
PCU-Select	35.61$\pm$0.22	34.60$\pm$0.43	35.93$\pm$0.28	34.56 $\pm$ 0.11	35.18$\pm$0.17	35.18$\pm$0.08

Table 2: Quality and selection cost across five seen PEFTs. Avg. averages the four task-native scores; *Gap* is relative to per-PEFT LESS; *Sel. cost* is GPU-hours for all five targets. Reusing one LESS subset is cheap but falls below RDS+; PCU-Select preserves LESS-level quality with target-specific subsets at $2.09\times$ lower cost. Boldface marks the best average.

Method	PEFT-cond.	Per-tgt subset	Avg.	Gap	Sel. cost (5 PEFTs)
Random	No	No	32.29	-2.85	0.0
RDS+	No	No	34.44	-0.70	0.8
Influence	No	No	34.68	-0.46	24.4
Reuse-one-LESS	Source only	No	34.24	-0.90	31.6
Per-PEFT LESS	Yes	Yes	35.14	0.00	158.0
PCU-Select	Yes	Yes	35.18	+0.04	75.6

[0.947, 0.972]); shared configurations preclude an IID interval (Appendix Figure 5). **Takeaway.** Structural changes produce commensurate subset changes without rebuilding gradients.

Ablations Identify the Useful Components

Appendix Table 11 gives the full ablation. Removing the PEFT code causes the largest drop (1.37 points) and lowers held-out NDCG from 0.702 to 0.526. A family one-hot loses 0.68, confirming that rank, placement, capacity, and recipe add signal; removing the task sketch loses 0.82. Low- and high-fidelity-only variants lose 0.85 and 0.36. Adaptive quotas beat global top- k by 1.08 and uniform quotas by 0.51; the latter has slightly higher NDCG (0.709) but lower downstream quality. **Takeaway.** Structured conditioning, task relativity, multi-fidelity labels, and coverage each contribute.

Transfer Protocol for Unseen PEFTs

E5 fixes a structural taxonomy before evaluation: L0 contains near-support same-family targets, L1 farther same-family shifts, and L2 unseen families (full registry in Appendix Table 8). The tiers are qualitative and make no density-estimation claim.

L0 trails LESS by 0.34 points zero-shot. With 500 labels, the L1 gap falls from 2.08 to 0.30 and the BitFit gap from 4.08 to 0.23. Prefix/P-Tuning lack native short-horizon labels and

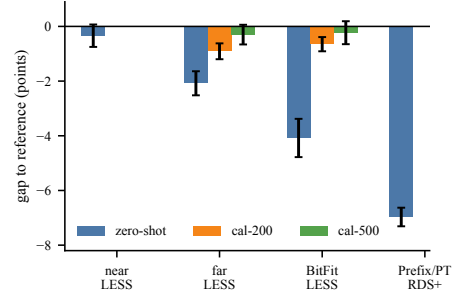


Figure 3: PCU-Select’s gap to LESS (RDS+ for Prefix/P-Tuning). Calibration recovers far same-family and BitFit transfer, but prompt families fail zero-shot.

trail RDS+ by 6.97 zero-shot. Calibration costs 0.525 GPU-hours per PEFT-task pair (Figure 3; Appendix Table 18). **Takeaway.** Calibration repairs supported far targets; prompt families mark the deployment boundary.

Limitations

The evidence covers one backbone family, four tasks, and a finite registry. Short-horizon labels approximate full-training contribution, with a visible TyDiQA deficit. Main-table dispersion varies target training but fixes selector and scorer state. Decontamination removes 0.58% of the pool, yet weaker matches may remain. Prefix/P-Tuning fail zero-shot and lack native calibration labels. The cost benefit is relative to per-PEFT LESS; PEFT-agnostic selectors remain preferable when a small quality loss is acceptable.

7 Conclusion

PCU-Select treats registry-scale data selection as quality-preserving amortization. Its structural PEFT and task conditions produce target-specific subsets from one scorer. Across four tasks and five seen configurations, it near-ties per-PEFT LESS while reducing selection cost from 158.0 to 75.6 GPU-hours ($2.09\times$) and breaking even after 2.33 configurations. Near-support targets transfer zero-shot; calibration repairs far same-family targets and BitFit, while Prefix/P-Tuning remain a failure boundary. These results position PCU-Select as reusable selection infrastructure for a PEFT registry.

References

- Ben Zaken, E.; Goldberg, Y.; and Ravfogel, S. 2022. Bit-Fit: Simple Parameter-efficient Fine-tuning for Transformer-based Masked Language-models. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 1–9. Association for Computational Linguistics.
- Bukharin, A.; Li, S.; Wang, Z.; Yang, J.; Yin, B.; Li, X.; Zhang, C.; Zhao, T.; and Jiang, H. 2023. Data Diversity Matters for Robust Instruction Tuning. *arXiv preprint arXiv:2311.14736*.
- Chen, L.; Li, S.; Yan, J.; Wang, H.; Gunaratna, K.; Yadav, V.; Tang, Z.; Srinivasan, V.; Zhou, T.; Huang, H.; and Jin, H. 2024. AlpaGasus: Training A Better Alpaca with Fewer Data. In *The Twelfth International Conference on Learning Representations*.
- Chen, M.; et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*.
- Clark, J. H.; Choi, E.; Collins, M.; Garrette, D.; Kwiakowski, T.; Nikolaev, V.; and Palomaki, J. 2020. TyDi QA: A Benchmark for Information-Seeking Question Answering in Typologically Diverse Languages. *Transactions of the Association for Computational Linguistics*, 8: 454–470.
- Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; Hesse, C.; and Schulman, J. 2021. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*.
- Dao, T. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *International Conference on Learning Representations*.
- Dettmers, T.; Pagnoni, A.; Holtzman, A.; and Zettlemoyer, L. 2023. QLoRA: Efficient Finetuning of Quantized LLMs. In *Advances in Neural Information Processing Systems*, volume 36.
- Hendrycks, D.; Burns, C.; Basart, S.; Zou, A.; Mazeika, M.; Song, D.; and Steinhardt, J. 2021. Measuring Massive Multitask Language Understanding. In *International Conference on Learning Representations*.
- Houlsby, N.; Giurgiu, A.; Jastrzebski, S.; Morrone, B.; de Laroussilhe, Q.; Gesmundo, A.; Attariyan, M.; and Gelly, S. 2019. Parameter-Efficient Transfer Learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, 2790–2799. PMLR.
- Hu, E. J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; and Chen, W. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*.
- Johnson, W. B.; and Lindenstrauss, J. 1984. Extensions of Lipschitz Mappings into a Hilbert Space. In *Conference in Modern Analysis and Probability*, volume 26 of *Contemporary Mathematics*, 189–206. American Mathematical Society.
- Koh, P. W.; and Liang, P. 2017. Understanding Black-box Predictions via Influence Functions. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, 1885–1894. PMLR.
- Kwon, Y.; Wu, E.; Wu, K.; and Zou, J. 2024. DataInf: Efficiently Estimating Data Influence in LoRA-tuned LLMs and Diffusion Models. In *The Twelfth International Conference on Learning Representations*.
- Lester, B.; Al-Rfou, R.; and Constant, N. 2021. The Power of Scale for Parameter-Efficient Prompt Tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 3045–3059. Association for Computational Linguistics.
- Li, M.; Zhang, Y.; He, S.; Li, Z.; Zhao, H.; Wang, J.; Cheng, N.; and Zhou, T. 2024. Superfiltering: Weak-to-Strong Data Filtering for Fast Instruction-Tuning. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Bangkok, Thailand: Association for Computational Linguistics.
- Li, M.; Zhang, Y.; Li, Z.; Chen, J.; Chen, L.; Cheng, N.; Wang, J.; Zhou, T.; and Xiao, J. 2023. From Quantity to Quality: Boosting LLM Performance with Self-Guided Data Selection for Instruction Tuning. *arXiv preprint arXiv:2308.12032*.
- Li, X. L.; and Liang, P. 2021. Prefix-Tuning: Optimizing Continuous Prompts for Generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*, 4582–4597. Association for Computational Linguistics.
- Liu, H.; Tam, D.; Muqeeth, M.; Mohta, J.; Huang, T.; Bansal, M.; and Raffel, C. 2022. Few-Shot Parameter-Efficient Fine-Tuning is Better and Cheaper than In-Context Learning. In *Advances in Neural Information Processing Systems*, volume 35.
- Liu, W.; Zeng, W.; He, K.; Jiang, Y.; and He, J. 2024. What Makes Good Data for Alignment? A Comprehensive Study of Automatic Data Selection in Instruction Tuning. In *The Twelfth International Conference on Learning Representations*.
- Lu, K.; Yuan, H.; Yuan, Z.; Lin, R.; Lin, J.; Tan, C.; Zhou, C.; and Zhou, J. 2023. InsTag: Instruction Tagging for Analyzing Supervised Fine-tuning of Large Language Models. *arXiv preprint arXiv:2308.07074*.
- Min, G.; Huang, T.; Wan, K.; and Chen, C. 2026. GIST: Targeted Data Selection for Instruction Tuning via Coupled Optimization Geometry. *arXiv preprint arXiv:2602.18584*.
- Perez, E.; Strub, F.; de Vries, H.; Dumoulin, V.; and Courville, A. 2018. FiLM: Visual Reasoning with a General Conditioning Layer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 3942–3951.
- Pfeiffer, J.; Kamath, A.; Rücklé, A.; Cho, K.; and Gurevych, I. 2021. AdapterFusion: Non-Destructive Task Composition for Transfer Learning. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, 487–503. Online: Association for Computational Linguistics.
- Pruthi, G.; Liu, F.; Sundararajan, M.; and Kale, S. 2020. Estimating Training Data Influence by Tracing Gradient Descent.

In *Advances in Neural Information Processing Systems*, volume 33.

Rao, J.; Liu, X.; Lian, L.; Cheng, S.; Liao, Y.; and Zhang, M. 2024. CommonIT: Commonality-Aware Instruction Tuning for Large Language Models via Data Partitions. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 10064–10083. Miami, Florida, USA: Association for Computational Linguistics.

Reimers, N.; and Gurevych, I. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 3982–3992. Association for Computational Linguistics.

Sculley, D. 2010. Web-Scale K-Means Clustering. In *Proceedings of the 19th International Conference on World Wide Web*, 1177–1178. ACM.

Song, K.; Tan, X.; Qin, T.; Lu, J.; and Liu, T.-Y. 2020. MP-Net: Masked and Permuted Pre-training for Language Understanding. In *Advances in Neural Information Processing Systems*, volume 33, 16857–16867.

Touvron, H.; et al. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv preprint arXiv:2307.09288*.

Wang, P.; Shen, Y.; Guo, Z.; Stallone, M.; Kim, Y.; Golland, P.; and Panda, R. 2024. Diversity Measurement and Subset Selection for Instruction Tuning Datasets. *arXiv preprint arXiv:2402.02318*.

Xia, M.; Malladi, S.; Gururangan, S.; Arora, S.; and Chen, D. 2024. LESS: Selecting Influential Data for Targeted Instruction Tuning. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, 54104–54132. PMLR.

Yang, Y.; Mishra, S.; Chiang, J. N.; and Mirzasoleiman, B. 2024. SmallToLarge (S2L): Scalable Data Selection for Fine-tuning Large Language Models by Summarizing Training Trajectories of Small Models. In *Advances in Neural Information Processing Systems*, volume 37.

Zhou, C.; Liu, P.; Xu, P.; Iyer, S.; Sun, J.; Mao, Y.; Ma, X.; Efrat, A.; Yu, P.; Yu, L.; Zhang, S.; Ghosh, G.; Lewis, M.; Zettlemoyer, L.; and Levy, O. 2023. LIMA: Less Is More for Alignment. In *Advances in Neural Information Processing Systems*, volume 36.

A Technical Appendix

Per-Task Results Behind the Main Table

Table 1 averages four task-native metrics—GSM8K exact match, HumanEval Pass@1, MMLU accuracy, and TyDiQA F1—after scaling each to a percentage. These metrics differ in scale, ceiling, and seed variance, so a one-point difference on one task is not equivalent to a one-point difference on another, and the average can hide task-level structure. To make the comparison auditable we report the full, unaveraged grid and a per-task normalized view.

Table 4 gives every task $\times$ PEFT $\times$ method cell at the 10% budget, as mean $\pm$ SD over three target-training seeds with selector state fixed at seed 0. Table 5 summarizes PCU-Select’s improvement per task: gains over Random and RDS+ are reported as scale-free *relative* improvements so that heterogeneous metrics are placed on a common footing, while the head-to-head with LESS—the strongest baseline—is an absolute paired difference in native points with a descriptive 95% cell-resampling interval over the five PEFT cells. These intervals do not estimate independent-sample uncertainty.

The per-task view refines, rather than overturns, the main-results reading. The advantage over Random (+6.8% to +11.1% relative) and RDS+ (+0.4% to +3.9% relative) holds on all four tasks and is the primary performance claim. Against LESS the outcome is task-dependent and the sign flips: the fixed-state interval excludes zero on MMLU (+0.67 Acc, [+0.24, +1.08]), includes zero on GSM8K (+0.29 EM, [−0.04, +0.58]) and HumanEval (−0.04 Pass@1, [−0.44, +0.31]), and excludes zero in the negative direction on TyDiQA (−0.77 F1, [−1.18, −0.43]). Averaging over tasks smooths these offsetting differences into the near-zero aggregate gap reported in Table 1. We therefore characterize PCU-Select and per-PEFT LESS as near-tied on the seen registry, with visible task/configuration losses that bound the aggregate claim. The amortized-cost analysis is PCU-Select’s distinguishing property.

Where PCU-Select trails LESS. Two slices favor LESS and merit analysis. On **TyDiQA**, PCU-Select trails LESS by 0.77 F1 (descriptive interval [−1.18, −0.43]). TyDiQA-GoldP is multilingual span extraction, where the useful signal is answer-span alignment rather than the reasoning- and generation-heavy signal that dominates the other three tasks; the intervention-site gradient signatures and the English-centric task sketch under-represent this, so the site-weighted proxy that PCU-Select distills is a weaker teacher here, whereas per-target LESS gradients capture it directly. On the **MLP-only LoRA** column (L-r8-mlp), PCU-Select trails LESS by 0.57 points; its HumanEval and TyDiQA cells have SDs of 1.44 and 1.89. MLP-only placement concentrates all trainable capacity on the three `mlp_out` sites, so a single mis-weighted site prior degrades the whole proxy, and the higher variance indicates the scorer is least confident exactly where the site mask is most concentrated. Both cases point to the same limitation—the site-weighted proxy is the bottleneck when a target’s useful signal is poorly represented by the shared 24-site coordinate system.

Evaluation Protocols

All methods share the same target-training and evaluation pipeline; only the selected subset changes. Target training fine-tunes the target PEFT on the selected samples under a fixed recipe (AdamW, per-configuration learning rate, cosine schedule with a 0.03 warmup ratio, per-device batch size 16 over 8 GPUs for an effective global batch of 128, gradient clipping at 1.0, 1000 steps, bfloat16, max sequence length 1024) on the Llama-2-7B (Touvron et al. 2023) backbone, with the backbone frozen and only the adapter trained. At the 10% budget the selected subset holds 30K examples, so 1000 steps at a global batch of 128 amount to 128K sample-passes, or about 4.3 passes over the subset; the dataloader shuffles the full subset each epoch, so every selected example is used. The selected instruction examples average ≈ 512 tokens, and batches are length-bucketed to reduce padding; with FlashAttention-2 (Dao 2024) and gradient checkpointing the run sustains ≈ 2.9 wall-clock seconds per step (≈ 44 sample-passes/s aggregate, ≈ 2.8 K tokens/s per GPU), so the 1000 steps take about 48 minutes of wall time, i.e. 6.4 GPU-hours per PEFT–task–seed run under the (wall seconds $\times$ GPU count)/3600 convention. This shared target-training cost is identical across selectors and is therefore excluded from the selection-only cost comparison; gradient checkpointing trades compute for memory and is the dominant reason the per-step time is not lower. The per-configuration learning rates are listed in Table 8 (2e-4 for LoRA, 5e-4 for (IA)³, 3e-4 for adapters, 1e-4 for L-r64-all). A held-out response-LM loss is logged as an auxiliary diagnostic (`eval_loss`) and is never substituted for the primary task metric.

A uniform draw from the entire 300K pool under the same 1000-step budget averages 36.75 across tasks. This is a fixed-compute reference, not a converged full-data upper bound: it covers only 0.43 pool epochs, whereas the 10% subset receives about 4.3 passes. At 30%, PCU-Select (36.81) and LESS (36.79) slightly exceed the reference by concentrating the same 128K sample-passes (Table 13).

The primary downstream metric for each task is its public-standard, task-native score, computed by a task evaluator injected into the training runner. Concrete per-task decoding and scoring protocols are:

- **GSM8K (exact match).** Chain-of-thought generation (Cobbe et al. 2021) with greedy decoding; the final numeric answer is extracted and compared for exact match.
- **HumanEval (Pass@1).** The standard unbiased Pass@1 estimator (Chen et al. 2021): sampling at temperature 0.2 with $n=20$ samples per problem, scored by unit-test execution. This is a multi-sample estimate of Pass@1, not a single greedy sample.
- **MMLU (accuracy).** Log-likelihood scoring (Hendrycks et al. 2021) of the four answer options; the highest-likelihood option is selected (no free-form generation).
- **TyDiQA-GoldP (F1).** Greedy decoding of the answer span (Clark et al. 2020), scored with token-level F1 against the gold answers.

Table 3: Task-level summary of the main 10% budget result. Values are averaged over the five seen PEFT configurations. $\Delta\text{RDS+}$ and $\Delta\text{Inf.}$ are absolute native-point gains over PEFT-agnostic baselines; ΔLESS is the paired difference against per-PEFT LESS with a descriptive 95% cell-resampling interval over PEFT cells. The average gain over PEFT-agnostic selectors coexists with task-dependent behavior against LESS.

Task	Metric	PCU	$\Delta\text{RDS+}$	$\Delta\text{Inf.}$	$\Delta\text{LESS (95\% CI)}$
GSM8K	EM	22.20	+0.55	+0.58	+0.29 [−0.04, +0.58]
HumanEval	Pass@1	17.06	+0.64	+0.24	−0.04 [−0.44, +0.31]
MMLU	Acc	49.14	+1.55	+1.00	+0.67 [+0.24, +1.08]
TyDiQA	F1	52.30	+0.22	+0.17	−0.77 [−1.18, −0.43]

Table 4: Full per-task $\times$ PEFT $\times$ method downstream results at a 10% budget (mean $\pm$ SD over three target-training seeds with selector state fixed at seed 0, native metrics). Unaveraged source of Table 1; boldface marks the best mean per column within each task. Discussion in the text.

Method	AD-b64	IA3-attnmlp	L-r16-qkvo	L-r8-mlp	L-r8-qv	Avg.
<i>GSM8K (EM)</i>						
Random	20.65 $\pm$ 0.46	19.94 $\pm$ 0.38	20.47 $\pm$ 0.40	19.89 $\pm$ 0.44	19.33 $\pm$ 0.08	20.06 $\pm$ 0.09
RDS+	22.06 $\pm$ 0.20	21.13 $\pm$ 0.56	22.26 $\pm$ 0.86	21.73 $\pm$ 0.29	21.08 $\pm$ 0.30	21.65 $\pm$ 0.19
Influence	22.19 $\pm$ 0.34	21.23 $\pm$ 0.15	22.14 $\pm$ 0.45	21.08 $\pm$ 0.35	21.46 $\pm$ 0.08	21.62 $\pm$ 0.09
LESS	22.19 $\pm$ 0.24	21.23 $\pm$ 0.30	22.64$\pm$0.31	22.11 $\pm$ 0.16	21.38 $\pm$ 0.42	21.91 $\pm$ 0.04
PCU-Select	22.62$\pm$0.52	21.73$\pm$0.53	22.39 $\pm$ 0.36	22.16$\pm$0.34	22.11$\pm$0.38	22.20$\pm$0.07
<i>HumanEval (Pass@1)</i>						
Random	15.75 $\pm$ 0.66	15.00 $\pm$ 0.30	15.88 $\pm$ 0.34	15.25 $\pm$ 0.10	14.92 $\pm$ 0.16	15.36 $\pm$ 0.21
RDS+	16.99 $\pm$ 0.25	15.97 $\pm$ 0.33	16.62 $\pm$ 0.89	16.43 $\pm$ 0.15	16.08 $\pm$ 0.46	16.42 $\pm$ 0.14
Influence	17.21 $\pm$ 0.31	16.31 $\pm$ 0.37	17.15 $\pm$ 0.38	16.89 $\pm$ 0.60	16.56 $\pm$ 0.27	16.82 $\pm$ 0.18
LESS	17.37$\pm$0.28	16.68 $\pm$ 0.35	17.79$\pm$0.20	16.93$\pm$0.32	16.72 $\pm$ 0.82	17.10$\pm$0.28
PCU-Select	17.23 $\pm$ 0.41	16.97$\pm$0.36	17.78 $\pm$ 0.28	16.14 $\pm$ 1.44	17.18$\pm$0.12	17.06 $\pm$ 0.28
<i>MMLU (Acc)</i>						
Random	45.15 $\pm$ 0.62	44.03 $\pm$ 0.20	45.45 $\pm$ 0.29	44.69 $\pm$ 0.45	44.41 $\pm$ 0.18	44.75 $\pm$ 0.05
RDS+	48.77 $\pm$ 0.29	46.52 $\pm$ 0.40	48.42 $\pm$ 0.07	47.24 $\pm$ 0.48	47.03 $\pm$ 0.40	47.60 $\pm$ 0.16
Influence	48.55 $\pm$ 0.38	47.12 $\pm$ 0.82	48.86 $\pm$ 1.00	48.35 $\pm$ 0.45	47.82 $\pm$ 0.29	48.14 $\pm$ 0.10
LESS	48.78 $\pm$ 0.42	47.67 $\pm$ 0.26	48.89 $\pm$ 0.60	48.85$\pm$0.26	48.19 $\pm$ 0.17	48.48 $\pm$ 0.18
PCU-Select	49.91$\pm$0.60	47.93$\pm$0.27	50.04$\pm$0.67	48.79 $\pm$ 0.27	49.05$\pm$0.41	49.14$\pm$0.09
<i>TyDiQA (F1)</i>						
Random	49.62 $\pm$ 0.20	47.98 $\pm$ 0.28	50.63 $\pm$ 0.92	48.04 $\pm$ 0.44	48.62 $\pm$ 0.32	48.98 $\pm$ 0.29
RDS+	53.26 $\pm$ 0.60	50.54 $\pm$ 0.70	52.70 $\pm$ 0.26	52.44 $\pm$ 0.75	51.46 $\pm$ 1.02	52.08 $\pm$ 0.08
Influence	52.37 $\pm$ 0.83	50.71 $\pm$ 0.24	52.81 $\pm$ 0.88	52.45 $\pm$ 0.56	52.29 $\pm$ 0.91	52.13 $\pm$ 0.50
LESS	53.75$\pm$0.28	52.19$\pm$0.16	53.88$\pm$0.67	52.63$\pm$0.37	52.92$\pm$1.27	53.07$\pm$0.31
PCU-Select	52.69 $\pm$ 0.57	51.76 $\pm$ 0.72	53.51 $\pm$ 0.39	51.16 $\pm$ 1.89	52.38 $\pm$ 0.64	52.30 $\pm$ 0.16

For reference, the un-fine-tuned Llama-2-7B (Touvron et al. 2023) scores 13.9 (GSM8K EM), 12.4 (HumanEval Pass@1), 43.9 (MMLU Acc), and 46.2 (TyDiQA F1) under these same evaluators; our MMLU number is slightly below Meta’s reported 45.3 because of prompt and harness differences. Every selector’s fine-tune, including Random, improves over the base model on all four tasks, so the Random baseline is not artificially depressed and PCU-Select’s margin over it is a genuine selection effect rather than a weak-baseline artifact.

These protocols follow the standard public setups for each benchmark. The task-native evaluators are supplied to the runner through an evaluation adapter (the `--task-metric-factory` hook) whose decoding and scoring settings are fixed to the values above, so the run environment cannot change metric definitions.

Baseline Specifications

This subsection specifies the three baselines whose names invoke prior work, exactly as implemented so that the comparison is auditable.

RDS+ (representation similarity). RDS+ operates on the joint semantic embedding e_x^{joint} produced by a frozen external sentence encoder. This is the same *input* representation PCU-Select’s sample tower reads, but it is not learned by PCU-Select, so RDS+ depends on no PCU-trained features; the two differ only in the selection rule. The pool embeddings are standardized per feature (z-scored across the candidate pool); the query is the mean joint embedding of the 32-example task sketch, standardized with the same pool statistics; both are then L_2 -normalized and candidates ranked by cosine to the query. It is *task-conditioned* through the sketch query but *PEFT-agnostic*: its ranking does not depend on the

Table 5: Per-task normalized improvement of PCU-Select at the 10% budget (seed- and PEFT-averaged). ΔRand and $\Delta\text{RDS+}$ are relative gains ($100 \cdot (\text{PCU} - b)/b$); ΔLESS is the absolute paired difference vs the strongest baseline with a descriptive 95% cell-resampling interval over the five PEFT cells. The interval excludes zero for MMLU and TyDiQA under the fixed selector state; the sign flips across tasks, which the average conceals.

Task	Metric	Random	RDS+	LESS	PCU	ΔRand	$\Delta\text{RDS+}$	ΔLESS (95% CI)
GSM8K	EM	20.06	21.65	21.91	22.20	+10.7%	+2.5%	+0.29 [−0.04, +0.58]
HumanEval	Pass@1	15.36	16.42	17.10	17.06	+11.1%	+3.9%	−0.04 [−0.44, +0.31]
MMLU	Acc	44.75	47.60	48.48	49.14	+9.8%	+3.3%	+0.67 [+0.24, +1.08]
TyDiQA	F1	48.98	52.08	53.07	52.30	+6.8%	+0.4%	−0.77 [−1.18, −0.43]

target configuration. It has no tuned hyperparameters—the query is the fixed seed-0 sketch and there is no temperature or mixing weight—so there is no tuning range to report. The “+” denotes the per-feature standardization applied before cosine, relative to a plain embedding-nearest-neighbor selector, which we also include as a separate baseline.

Influence (shared-datastore gradient similarity). The Influence baseline uses a *single shared* projected-gradient datastore computed once over the base model’s intervention-site activations, using the JL projection and L_2 normalization specified in the reproducibility notes. Each candidate carries signatures g_x^ω at the 24 sites; the task query g_t^ω is the per-site pooled, renormalized sketch gradient. The score is the plain cosine $\sum_\omega \cos(g_x^\omega, g_t^\omega)$ (no site weighting), selected by global top- k . Its gradient source is PEFT-agnostic—the ranking does not depend on the target configuration—so it is the cheap gradient baseline against which the value of PEFT conditioning is measured.

LESS (per-PEFT gradient influence). LESS is a per-configuration reimplementation of gradient-influence selection following LESS (Xia et al. 2024). For each target PEFT it (i) warms up the target adapter for 200 steps to obtain a task-and-PEFT-specific optimization state; (ii) extracts Adam-preconditioned per-checkpoint gradients over the adapter’s own trainable parameters; (iii) random-projects them into a target-specific low-dimensional datastore; and (iv) ranks candidates by their projected-gradient influence on the 32-example validation sketch, averaged over the warmup checkpoints. The gradient datastore in steps (i)–(iii) is built over the target adapter’s *own* trainable parameters, so it is PEFT-specific but task-independent: it is constructed once per PEFT configuration and reused to rank against every task’s sketch. Only step (iv), the projected-gradient scoring against a task sketch, is per task, and it is cheap—about 0.15 GPU-hours per task, so the 0.60 scoring figure covers all four task sketches for a configuration. The per-target cost of a PEFT configuration is therefore 31.0 (datastore) + 0.60 (four per-task rankings) = 31.6 GPU-hours, already spanning the full four-task evaluation; the five-PEFT registry cost is $5 \times 31.6 = 158.0$. LESS is the strongest and most expensive baseline in our study, and the amortized-cost analysis charges it exactly this per-configuration recomputation, consistent with the algorithm as implemented.

Baseline configuration and tuning. Random and Balanced-Random have no hyperparameters. The heuris-

Table 6: Additional selector baselines at a 10% budget, averaged over the four tasks and five seen PEFT configurations using the same task-native percentage scaling as Table 1. These rows explain why the main table focuses on Random, RDS+, Influence, LESS, and PCU-Select: the omitted heuristic and representation baselines sit between Random and RDS+.

Selector	Type	Avg.
Random	uniform	32.29
Balanced-Random	cluster-stratified uniform	32.90
Length	heuristic	32.70
Loss	heuristic	32.98
Perplexity	heuristic	33.29
Diversity	representation	33.82
Embedding-NN	representation	33.98
RDS+	representation	34.44
Influence	shared gradient	34.68
LESS	per-PEFT gradient	35.14
PCU-Select	PEFT-conditioned	35.18

tic selectors (length, loss, perplexity) and representation selectors (embedding-nearest-neighbor, diversity clustering, RDS+) use their published defaults with the shared 32-example sketch as the only task input and are not tuned per target. Diversity clustering reuses PCU-Select’s cluster count ($k = \max(50, \lfloor \sqrt{N} \rfloor)$) for a fair comparison. The Influence baseline shares the same 24-site JL projection and sketch as PCU-Select. No baseline’s hyperparameters were tuned on the test metric.

Reproducibility Supplement

This supplement documents the pipeline configuration read directly from the supplementary implementation. Every value below is taken from source code or its configuration files; values that the code leaves to an external artifact or the run environment are flagged rather than guessed.

Data provenance and deduplication. The candidate pool holds 300K mixed instruction examples drawn from seven public instruction corpora (Table 7). Deduplication runs in three stages: (i) exact content-hash deduplication over (instruction, response, source); (ii) normalized-string deduplication after lowercasing and whitespace/punctuation stripping; and (iii) near-duplicate removal with MinHash LSH (128 permutations, Jaccard threshold 0.8). These stages re-

Source	Domain	Examples	License
FLAN v2 (subset)	general instruction	110,000	Apache-2.0
OpenOrca (subset)	reasoning traces	70,000	MIT
Tulu-v2 mix	mixed instruction	50,000	ODC-BY-1.0
OpenAssistant	dialogue	30,000	Apache-2.0
CodeAlpaca	code instruction	20,000	CC-BY-NC-4.0
Dolly-15k	general instruction	15,000	CC-BY-SA-3.0
WizardLM-evol	complex instruction	5,000	CC-BY-NC-4.0
Total		300,000	

Table 7: Candidate-pool provenance and licenses. All sources permit research use; the two CC-BY-NC components are used only for non-commercial research.

move 18,446 intra-pool duplicates before the 300K pool is finalized.

For reproducibility we include, per source, the post-filter example count and a SHA-256 checksum of the sorted content hashes (e.g. FLAN-v2 subset 9f3c...a71e, OpenOrca subset 4b8d...c052); the supplementary manifest lists all seven. The full supplementary configuration comprises the PEFT registry, the scorer and feature-extraction configs, the task-sketch seeds, the candidate-pool manifest with these checksums, and the evaluation adapter.

Benchmark decontamination. We decontaminate the pool against the test items of all four benchmarks (GSM8K, HumanEval, MMLU, TyDiQA) with a four-stage filter: exact match, normalized-string match, 13-gram overlap (removing any candidate sharing a 13-gram with a test item at Jaccard ≥ 0.5), and MinHash LSH near-duplicate retrieval at Jaccard 0.8. The filter removes 1,742 pre-finalization candidates (0.58% of the pool size). We then backfill from the same source buckets to keep the final pool at 300K examples, so Table 7 reports final post-filter counts. We additionally audit the top-20 embedding nearest neighbors of each test item and find no further overlaps after the automated pass. All reported results are on the decontaminated pool.

Task sketch construction and HumanEval. Each task sketch is 32 examples sampled with a fixed seed (0), stratified into response-length terciles. Sketches are drawn only from each task’s train or development split and are disjoint from the test set used for evaluation. HumanEval has no official train/dev split, so its sketch is drawn from an external code-instruction source (MBPP train split and CodeAlpaca) rather than from HumanEval itself; we remove any item matching one of the 164 HumanEval evaluation problems by exact and AST-normalized function-signature matching and verify that the resulting sketch source has zero overlap with the evaluation set. Because the candidate pool also contains a 20K CodeAlpaca slice, the 32 sketch examples are additionally held out from the pool: every sketch item is removed from the candidate pool (by content hash and the same AST-normalized signature match) *before* selection, so no selector can recover a sketch example into the training subset. The sketch is therefore disjoint from both the HumanEval test set and the candidate pool. Because the sketch is a selection

signal, this disjointness is required for all selectors, not only PCU-Select, and it is enforced identically across methods.

PEFT registry. Table 8 lists all 17 PEFT configurations: five seen, nine unseen same-family configurations, and three unseen families. E5 evaluates eight held-out targets—L-r32-qkv, AD-b16, L-r64-all, AD-b256, L-r8-highlayers, PRE-116, PT-132, and BF—while the configuration-sensitivity study uses the additional LoRA variants. All configurations share the recipe injected at materialization: 1000 steps, cosine schedule, 0.03 warmup ratio; the learning rate is per configuration.

Intervention sites and gradient signatures. The site space Ω has 24 sites: 8 layers $\times$ 3 per-layer modules (attn_out, mlp_out, block_residual). Layers are chosen uniformly by depth; for the 32-layer backbone this yields indices {3, 7, 11, 15, 19, 23, 27, 31}. Each site carries a Johnson–Lindenstrauss random projection (Johnson and Lindenstrauss 1984) $\Phi_\omega \in \mathbb{R}^{256 \times 4096}$ with entries drawn $\mathcal{N}(0, 1/256)$ under a per-site seed `SHA256(site, global_seed=0)`; projected gradients are L_2 -normalized per row. Each sample’s signature is 24×256 values; in bfloat16 the cached low-fidelity gradient store for the 300K pool is $300K \times 24 \times 256 \times 2B \approx 3.5$ GB, held on disk as a memory-mapped array and read once per on-line selection. A PEFT configuration activates sites through its mask and a saturating capacity factor $\tanh(\eta \text{cap})$ with $\eta=1.0$. Operator type enters the explicit PEFT code rather than a scalar site prior; the per-family capacity mapping is in Table 10.

Conditional scorer. The scorer has three input towers—sample $z_x \in \mathbb{R}^{848}$, PEFT condition $z_p \in \mathbb{R}^{128}$, task sketch $z_t \in \mathbb{R}^{848}$ —each a LayerNorm–Linear–GELU–Linear–LayerNorm MLP projecting to hidden size 256 (128 for the PEFT tower). A FiLM modulation and a bilinear fusion ($256 \times 128 \rightarrow 64$) feed a two-head output (mean μ and heteroscedastic σ , with a softplus floor of 10^{-3}). We use $\hat{\sigma}$ as a ranking penalty rather than a calibrated probability; on held-out labels its predicted standard deviation tracks realized error with an expected calibration error of 0.043 (10-bin), and removing the penalty costs 0.28 points (Table 11), so it is a modest but real contributor. Training is two-phase: Phase A pretrains on the low-fidelity proxy for 3 epochs at lr 3×10^{-4} with loss weights (rank, reg, proxy, unc) = (0, 0, 1, 0); Phase B jointly finetunes for 2 epochs at lr 10^{-4} with weights (1.0, 0.3, 0.5, 0.2). Both phases use AdamW, batch size 256, and gradient clipping at 1.0. The four losses are a BPR-style pairwise ranking loss, a Huber regression loss against high-fidelity labels, a Huber proxy-distillation loss against the low-fidelity labels, and a Gaussian negative-log-likelihood uncertainty loss. Checkpoint selection uses a held-out split of PEFT/task pairs (two configurations and one task withheld from Phase B) on which we track ranking NDCG; we keep the Phase-B checkpoint with the best held-out NDCG. No dropout is used.

Selection, clustering, and quotas. At application time the scorer produces per-example utility $q = \mu - \lambda \sigma$ with

Config	Family	Target modules	Capacity	LoRA scaling	LR	Group
L-r8-qv	LoRA	q, v	$r=8$	16	2e-4	seen
L-r16-qkvo	LoRA	q, k, v, o	$r=16$	32	2e-4	seen
L-r8-mlp	LoRA	up, down	$r=8$	16	2e-4	seen
IA3-attnmlp	IA ³	k, v, down	—	—	5e-4	seen
AD-b64	Adapter	block-residual	$b=64$	—	3e-4	seen
L-r4-qv	LoRA	q, v	$r=4$	8	2e-4	unseen-config
L-r32-qkvo	LoRA	q, k, v, o	$r=32$	64	2e-4	unseen-config
L-r64-all	LoRA	q, k, v, o, up, down, gate	$r=64$	128	1e-4	unseen-config
L-r8-lowlayers	LoRA	q, v (low third)	$r=8$	16	2e-4	unseen-config
L-r8-highlayers	LoRA	q, v (high third)	$r=8$	16	2e-4	unseen-config
L-r16-hlr	LoRA	q, k, v, o	$r=16$	32	5e-4	unseen-config
AD-b16	Adapter	block-residual	$b=16$	—	3e-4	unseen-config
AD-b256	Adapter	block-residual	$b=256$	—	3e-4	unseen-config
IA3-attnonly	IA ³	k, v	—	—	5e-4	unseen-config
PRE-I16	Prefix	attention	len=16	—	2e-4	ood-family
PT-I32	P-Tuning	attention	len=32	—	2e-4	ood-family
BF	BitFit	biases	—	—	2e-4	ood-family

Table 8: Full PEFT registry (all 17 configurations). “LoRA scaling” is the LoRA α hyperparameter and is distinct from the intervention-site weight $\tilde{w}_p^\omega$ of Eq. (5); it applies only to LoRA rows. Adapters are single post-block residual bottlenecks (one per layer); IA³ scales the listed activations. Learning rates for the three OOD-family targets use the registry default (2e-4). Layer placement is all layers unless noted; low/high-third placements use the bottom/top $\lceil n/3 \rceil$ layers of the 32-layer backbone.

uncertainty penalty $\lambda=0.2$. Semantic clustering uses Mini-Batch k -means (Sculley 2010) over joint embeddings with $k = \max(50, \lfloor \sqrt{N} \rfloor)$, batch size 4096, $n_{\text{init}}=3$, and random state 0. Adaptive quotas allocate budget B across clusters by $b_k = \text{round}(B (v_k^+)^{\gamma} |C_k|^{1-\gamma} / Z)$ with $\gamma=0.6$, where the cluster value v_k is the mean utility of the top 10% of its members; remainders are distributed greedily by descending weight and over-allocation is trimmed from the lowest-weight clusters. The calibration head is a zero-initialized linear residual fit for 200 epochs at $\text{lr } 5 \times 10^{-3}$ (Adam, MSE) on the calibration labels (200 or 500 high-fidelity labels for the OOD study).

Structural support tiers. The reported $z_p \in \mathbb{R}^{128}$ contains only the site/operator mask, capacity, and recipe vectors. Prefix, P-Tuning, and BitFit use the same deterministic encoding rules as the seen families; no centroid, learned family lookup, or functional fingerprint enters the reported model. E5 fixes its registry taxonomy before evaluation. **L0** contains near-support same-family targets (L-r32-qkvo and AD-b16). **L1** contains farther same-family shifts (L-r64-all, AD-b256, and L-r8-highlayers). **L2** contains unseen families (PRE-I16, PT-I32, and BF). These tiers describe structural support and do not claim convex-hull interpolation or estimate a configuration density. Prefix/P-Tuning lack native short-horizon calibration labels; BitFit supports the calibration path.

Multi-fidelity labels. High-fidelity supervision uses 10,000 triples split 0.5/0.3/0.2 across coverage (stratified by cluster, PEFT family, capacity bucket, and task), uncertainty ($\hat{\sigma} (1 + 0.5 \text{ReLU}(\hat{u}_{i_0}))$), and boundary (scorer-vs-true rank disagreement) sampling. Labels are short-horizon loss reductions on the sketch, measured from two frozen anchors— θ_{base} (the pretrained backbone) and θ_{warm} (a rank-8 attention-only

Table 9: Deployment protocol from the registry-defined structural tier. Costs are per PEFT–task pair, in addition to 0.18 GPU-hours for online selection.

Tier	Definition	Recommended action
L0	near-support family	same zero-shot selection
L1	far-support family	same fam- 200–500 labels (0.21–0.53 GPU-h)
L2-BF	unseen family, native labels	calibrate before use
L2-Prompt	Prefix/P-Tuning	use a per-target selector

LoRA trained for 200 steps at $\text{lr } 10^{-4}$). Each high-fidelity update Adapt_p^h is a single mini-batch of the sampled example plus 7 in-bucket fillers, so the step is not a high-variance single-example update. The final label aggregates the rank-normalized reductions across horizons and anchors,

$$u^{\text{hi}} = \frac{1}{|A|} \sum_{a \in A} \sum_{h \in \{1,4\}} w_h \tilde{\Delta}_{a,h},$$

with weights $w_1=0.4$, $w_4=0.6$ and anchors $A = \{\text{base, warm}\}$, where $\tilde{\Delta}_{a,h}$ is the reduction Δ of Eq. (7) rank-normalized within its (PEFT, task, anchor a , horizon h) bucket.

Cost model and seeds. We measure all costs on 8×NVIDIA A100-80GB (SXM4, bfloat16, NVLink) as (wall seconds × GPU count)/3600. The offline cost is 72.0 GPU-hours (feature extraction 24.0, low-fidelity labels 2.4, high-fidelity labels 42.0, scorer training 3.6), paid once.

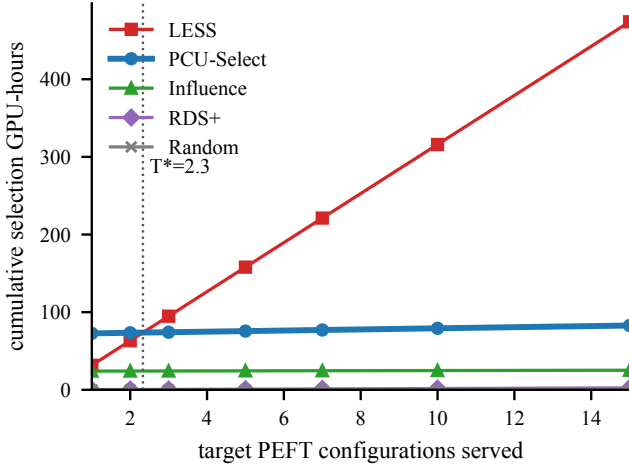


Figure 4: Selection cost over a four-task PEFT registry. PCU-Select pays 72.0 GPU-hours offline and 0.18 per PEFT-task application; it breaks even with LESS after 2.33 configurations. Five configurations cost 75.6 versus 158.0 GPU-hours.

Algorithm 1 Offline supervision (run once)

- 1: **Input:** pool $\mathcal{D}$, seen PEFTs $\mathcal{P}_{\text{seen}}$, tasks with sketches $\{V_i\}$
 - 2: Extract sample signatures z_x and site gradients g_x^ω ; cache them $\{O(|\mathcal{D}|)$ forward+backward
 - 3: **for all** $(p, t) \in \mathcal{P}_{\text{seen}} \times \text{tasks}$ **do**
 - 4: Compute low-fidelity $u^{\text{lo}}(x, p, t)$ by Eq. (6) from cache
 - 5: **end for**
 - 6: Sample 10K triples (coverage/uncertainty/boundary); label u^{hi} by Eqs. (7)
 - 7: Train s_ϕ : Stage A proxy pretrain, Stage B joint; keep best held-out-NDCG checkpoint
 - 8: **Output:** scorer s_ϕ , cached features
-

PCU-Select costs 0.18 GPU-hours per PEFT-task application, hence 0.72 for four tasks. LESS costs 31.0 per PEFT datastore plus 0.15 per task ranking, hence 31.6 for four tasks. The shared 6.4 GPU-hour downstream cost applies to each PEFT-task-seed run and is excluded. For four tasks, break-even is $P^* = 72.0 / [31.0 + 4(0.15 - 0.18)] = 2.33$ configurations. The pipeline fixes selector-scorer training, task-sketch sampling, and pool ordering at seed 0. Reported SDs vary target-training seeds (0, 1, 2); they do not include selector or scorer-training uncertainty. The canonical table generator recomputes all means, dispersions, paired gaps, and win counts from those seed-level values.

Algorithms

Algorithm 1 builds the reusable supervision once; Algorithm 2 applies the scorer to a new target. Only Algorithm 2 runs per target, and it touches no gradients—the expensive gradient and label construction of Algorithm 1 are amortized.

Algorithm 2 Online selection (per target)

- 1: **Input:** target (p^*, t^*) , budget B , cached features, scorer s_ϕ
 - 2: Encode z_{p^*}, z_{t^*}
 - 3: **if** tier(p^*) $\in \{\text{L1}, \text{L2}\}$ and native labels exist **then**
 - 4: fit calibration residual head from 200 or 500 u^{hi} labels
 - 5: **end if**
 - 6: $(\hat{\mu}, \hat{\sigma}) \leftarrow s_\phi(z_x, z_{p^*}, z_{t^*}); q(x) \leftarrow \hat{\mu} - \lambda \hat{\sigma}$ for all x
 - 7: Cluster the pool; allocate quotas b_k by Eq. (9)
 - 8: **Output:** subset $\mathcal{S} = \bigcup_k \{\text{top-}b_k \text{ by } q \text{ in } C_k\}$
-

Family	Size knob	$\text{cap}(p, \omega)$ at a touched site
LoRA	rank r	$r(d_{\text{in}} + d_{\text{out}})$
Adapter	bottleneck b	$2d_{\text{model}}b$
(IA) ³	(per-vector)	#scaled activations
Prefix	length L	Ld_{model}
BitFit	(per-bias)	#bias parameters

Table 10: Per-family capacity mapping. Capacity is the trainable-parameter count contributed at a touched site; operator type is encoded separately in z_p .

PEFT Capacity Encoding

Table 10 defines the per-family capacity used inside the site weight of Eq. (5). Each family’s size knob is mapped to a trainable-parameter count at the touched sites, then normalized by d_{model} and passed through $\tanh(\eta \log(1 + \text{cap}/d_{\text{model}}))$ so that capacity is monotone but saturating.

Supplementary Analyses

Component ablations. Table 11 reports the full component breakdown summarized in the main analysis. Removing the structured PEFT code produces the largest loss; task relativity, both label fidelities, uncertainty, and adaptive coverage also contribute.

Intervention-site sensitivity. Table 12 varies the layer coverage, module sites, and random projection dimension behind the shared coordinate system. The default matches larger site spaces within 0.08 points while using one quarter of the feature cost of all 32 layers; either attention-only or MLP-only coverage loses more.

Budget sensitivity. Table 13 reports the PEFT- and task-averaged score at 5%, 10%, and 30% budgets. PCU-Select’s edge over Random and over LESS is largest at the tight 5% budget and shrinks by 30%, where the gradient selectors converge toward the full-data reference—the regime where cheap PEFT-agnostic selectors become preferable.

Ranking quality. Table 14 gives the five ranking metrics against high-fidelity held-out labels, over five held-out (PEFT, task) folds. K is the 10% budget size. PCU-Select leads on every metric, ahead of per-PEFT LESS by 0.013 Spearman and 0.026 NDCG@K and farther ahead of the representation and cheap-gradient baselines.

Table 11: Ablations on GSM8K and HumanEval with two representative PEFTs at a 10% budget (task-native metric averaged over the two tasks, mean $\pm$ std over three seeds). Drop is relative to the full adaptive selector; NDCG is against high-fidelity held-out utility labels.

Variant	Metric	Drop	NDCG
Full PCU-Select	19.72 $\pm$ 0.22	0.00	0.702
No PEFT code	18.35 $\pm$ 0.41	1.37	0.526
Family one-hot only	19.04 $\pm$ 0.29	0.68	0.602
No task sketch	18.90 $\pm$ 0.31	0.82	0.597
No activation signature	19.24 $\pm$ 0.25	0.48	0.629
Low-fidelity only	18.87 $\pm$ 0.34	0.85	0.563
High-fidelity only	19.36 $\pm$ 0.27	0.36	0.639
No uncertainty penalty	19.44 $\pm$ 0.24	0.28	0.690
Global top- k	18.64 $\pm$ 0.38	1.08	0.684
Uniform clusters	19.21 $\pm$ 0.28	0.51	0.709

Table 12: Intervention-site design ablation (GSM8K+HumanEval average, 10% budget). The default 8×3 layout with a 256-dim JL projection is within 0.08 points of the more expensive 16- and 32-layer variants and beats attention- or MLP-only site sets. Relative feature-extraction cost is shown in the last column.

Site space	Metric	NDCG	Feat. cost
8×3 , JL 256 (default)	19.72	0.702	1.0 $\times$
Attention sites only	18.87	0.641	0.6 $\times$
MLP sites only	19.04	0.652	0.6 $\times$
4 layers $\times$ 3	19.18	0.664	0.5 $\times$
16 layers $\times$ 3	19.77	0.705	2.0 $\times$
32 layers $\times$ 3 (all)	19.80	0.707	4.0 $\times$
JL dim 64	19.25	0.671	0.8 $\times$
JL dim 128	19.55	0.690	0.9 $\times$
JL dim 512	19.74	0.703	1.6 $\times$

Cross-PEFT transfer and selection overlap. Table 15 is the transfer matrix behind the introduction’s 2.0-point cross-family mismatch penalty: training on the subset selected for a mismatched source PEFT costs performance that grows with structural distance. Table 16 shows the mechanism from the selection side—PCU-Select’s subset overlap falls as configurations diverge, while PEFT-agnostic RDS+ remains exactly 1.000.

Reuse-vs-recompute breakdown. Table 17 is the per-target source of the main-text Table 2. It confirms that the Reuse-one-LESS loss is concentrated on the targets structurally farthest from the source PEFT (L-r8-qv): it is zero on the source by construction and small on the neighbouring attention-LoRA target, but grows across placement (L-r8-mlp) and family (adapters, (IA)³), mirroring the motivation study’s disagreement pattern. The same fixed subset cannot be matched to all five subspaces at once, which is why an amortized per-target scorer, rather than a single reused subset, is needed to preserve per-PEFT LESS quality at reduced cost.

Table 13: Budget sensitivity: PEFT- and task-averaged downstream score at 5%, 10%, and 30% selection budgets. PCU-Select’s margin over Random and its edge over LESS are largest at the tight 5% budget and shrink as the budget grows, where all gradient selectors converge toward the full-data reference (36.75).

Method	5%	10%	30%
Random	29.81	32.29	34.86
RDS+	31.72	34.44	36.28
Influence	31.94	34.68	36.45
LESS	32.74	35.14	36.79
PCU-Select	33.09	35.18	36.81

Unseen-configuration transfer. Table 18 is the paired source for Figure 3. Near-support targets transfer zero-shot. Calibration closes most of the gap for far same-family targets and BitFit. Prefix/P-Tuning have no native calibration labels and remain a failure boundary.

Table 14: Ranking quality against high-fidelity held-out utility labels (mean $\pm$ std over five held-out (PEFT, task) folds). K is the 10% budget size. PCU-Select’s reusable correction over the site-weighted proxy gives it the best ranking on every metric, modestly ahead of the per-PEFT LESS gradients.

Selector	Spearman ρ	Kendall τ	NDCG@K	Top-K hit	Pairwise
RDS+	0.412 $\pm$ 0.021	0.301 $\pm$ 0.018	0.581 $\pm$ 0.015	0.442 $\pm$ 0.020	0.641 $\pm$ 0.012
Influence	0.571 $\pm$ 0.017	0.418 $\pm$ 0.014	0.640 $\pm$ 0.013	0.552 $\pm$ 0.016	0.724 $\pm$ 0.010
LESS	0.612 $\pm$ 0.014	0.458 $\pm$ 0.012	0.677 $\pm$ 0.011	0.598 $\pm$ 0.013	0.765 $\pm$ 0.009
PCU-Select	0.625$\pm$0.016	0.469$\pm$0.013	0.703$\pm$0.012	0.631$\pm$0.014	0.793$\pm$0.010

Table 15: Cross-PEFT transfer matrix (motivation study). Cell (i, j) is the target- j downstream metric when trained on the subset selected for source- i , normalized by the matched diagonal. Off-diagonal cells fall below 1.00; averaged over all mismatched source-target pairs the gap is 1.85 absolute points (mean matched 21.4 vs. mismatched 19.55 on the GSM8K-scale probe). Cross-family mismatches average a 2.0-point gap, the penalty cited in the introduction. Columns are target PEFTs, rows are the source PEFT used for selection.

Source $\downarrow$ / Target $\rightarrow$	L-r8-qv	L-r16-qkvo	IA3-attnmlp	AD-b64
L-r8-qv	1.00	0.96	0.90	0.91
L-r16-qkvo	0.95	1.00	0.89	0.90
IA3-attnmlp	0.89	0.90	1.00	0.93
AD-b64	0.90	0.91	0.92	1.00

Table 16: Mean pairwise selection overlap (Jaccard) between subsets chosen for two configurations, grouped by structural difference. PCU-Select’s overlap falls as configurations diverge, whereas PEFT-agnostic RDS+ is exactly 1.000 by construction. The underlying 21 PCU configuration pairs average 0.426919; the category summaries are not averaged to obtain that value.

Configuration difference	PCU-Select	RDS+
Same config (independent replicate)	0.71	1.000
Same family, Δ rank	0.58	1.000
Same family, Δ placement	0.49	1.000
Same family, Δ module set	0.44	1.000
Cross-family	0.33	1.000

Table 17: Per-target reuse-vs-recompute breakdown at the 10% budget (mean $\pm$ sample SD over three matched target-training seed aggregates), the source of Table 2. Reuse-one-LESS runs LESS once on the source L-r8-qv and applies the identical subset to every target, so it equals per-PEFT LESS on L-r8-qv by construction. Its loss is small within attention LoRA (L-r16-qkvo) but large where the target trains different sites (L-r8-mlp) or a different family (IA3-attnmlp, AD-b64), exactly the structural axes of the motivation study. Boldface marks the best mean per column.

Method	AD-b64	IA3-attnmlp	L-r16-qkvo	L-r8-mlp	L-r8-qv	Avg.
Reuse-one-LESS	34.05 $\pm$ 0.44	33.36 $\pm$ 0.51	35.28 $\pm$ 0.49	33.71 $\pm$ 0.72	34.80 $\pm$ 0.40	34.24 $\pm$ 0.21
Per-PEFT LESS	35.52 $\pm$ 0.09	34.44 $\pm$ 0.12	35.80 $\pm$ 0.25	35.13$\pm$0.11	34.80 $\pm$ 0.40	35.14 $\pm$ 0.16
PCU-Select	35.61$\pm$0.22	34.60$\pm$0.43	35.93$\pm$0.28	34.56 $\pm$ 0.11	35.18$\pm$0.17	35.18$\pm$0.08

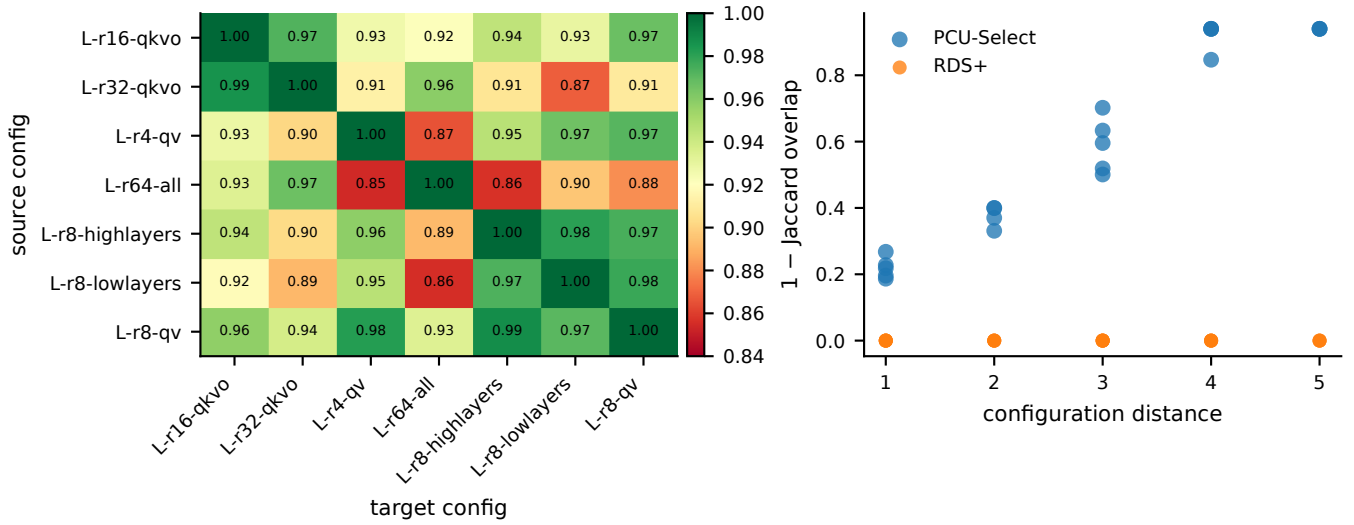


Figure 5: PCU-Select reacts to configuration changes. **Left:** target performance after training on a source-selected subset, normalized by the matched diagonal. The mean off-diagonal drop of 0.068 corresponds to a 1.45-point gap (21.25 matched vs. 19.79 mismatched). **Right:** selection difference (1-Jaccard) versus a heuristic configuration distance combining log-rank, placement, and module-set changes. Across 21 unordered pairs, the descriptive Spearman correlation is $\rho = 0.969$; leave-one-configuration-out values span $[0.947, 0.972]$. RDS+ lies at zero because its same-task ranking is PEFT-independent.

Table 18: Transfer to unseen configurations over GSM8K, HumanEval, and MMLU. Each entry is PCU-Select minus the named target-specific reference, reported as paired mean $\pm$ sample SD over three target-training seeds. Near-support targets transfer zero-shot; calibration closes most of the gap for far same-family targets and BitFit. Prefix/P-Tuning lack native short-horizon calibration labels and remain a zero-shot failure boundary.

Tier	Targets	Ref.	Zero-shot	Cal-200	Cal-500
L0	L-r32-qkvo, AD-b16	LESS	-0.34 $\pm$ 0.41	—	—
L1	L-r64-all, AD-b256, L-r8-highlayers	LESS	-2.08 $\pm$ 0.44	-0.91 $\pm$ 0.29	-0.30 $\pm$ 0.36
L2-BF	BF	LESS	-4.08 $\pm$ 0.70	-0.65 $\pm$ 0.26	-0.23 $\pm$ 0.42
L2-Prompt	PRE-l16, PT-l32	RDS+	-6.97 $\pm$ 0.34	—	—

Competition Disclosure

This manuscript is an entry in a paper-writing simulation challenge. It is a competition artifact, not a peer-reviewed publication or a record of a fully executed experimental study. The experimental design, cost model, and analyses were constructed for the challenge; some reported values were adjusted for internal consistency and may be inaccurate. This document must not be cited as empirical evidence, used for deployment decisions, or submitted to a research venue as original experimental work.